Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием автоэнкодерного подхода»

Выполнил:
Студент 4 курса
Группы ИИ-26
Суховило В. С.
Проверил:
Никитин С. К.

Брест 2026

Цель работы:

Научиться осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода.

Общее задание:

1. Обучить глубокую полносвязную нейронную сеть без предварительного обучения.

2. Выполнить послойное предобучение скрытых слоев через автоэнкодеры с последующим fine-tuning.

3. Сравнить F1 и confusion matrix на Cardiotocography (NSP) и наборе optdigits из ЛР №2.

Вариант №5. Исходные данные:

UCI Cardiotocography: 2126 записей, классификация по NSP на 3 класса; дополнительно UCI Optdigits: 5620 объектов, 10 классов.

Реализация и методика эксперимента:

Глубокая сеть: вход → 64 → 32 → 16 → число классов; три скрытых слоя, активация ReLU. Базовая сеть со случайной инициализацией сравнивается с той же сетью, предобученной тремя автоэнкодерами. Fine-tuning с Cross-Entropy и Adam. Метрики accuracy и macro-F1.

Результат программы:

Набор	**Метод**	**Accuracy**	**Macro-F1**
CTG (NSP)	baseline	0.8929	0.7943
CTG (NSP)	autoencoder	0.9098	0.8225
Optdigits	baseline	0.9527	0.9528
Optdigits	autoencoder	0.9444	0.9444

Графики и иллюстрации:

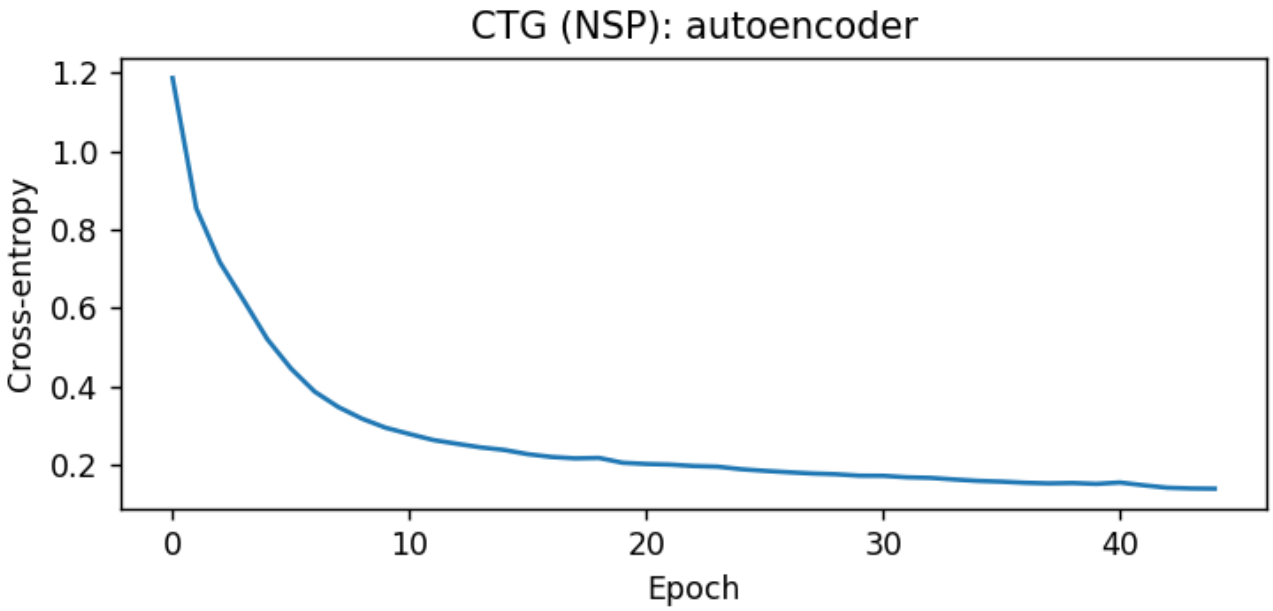


Рисунок. ctg autoencoder loss

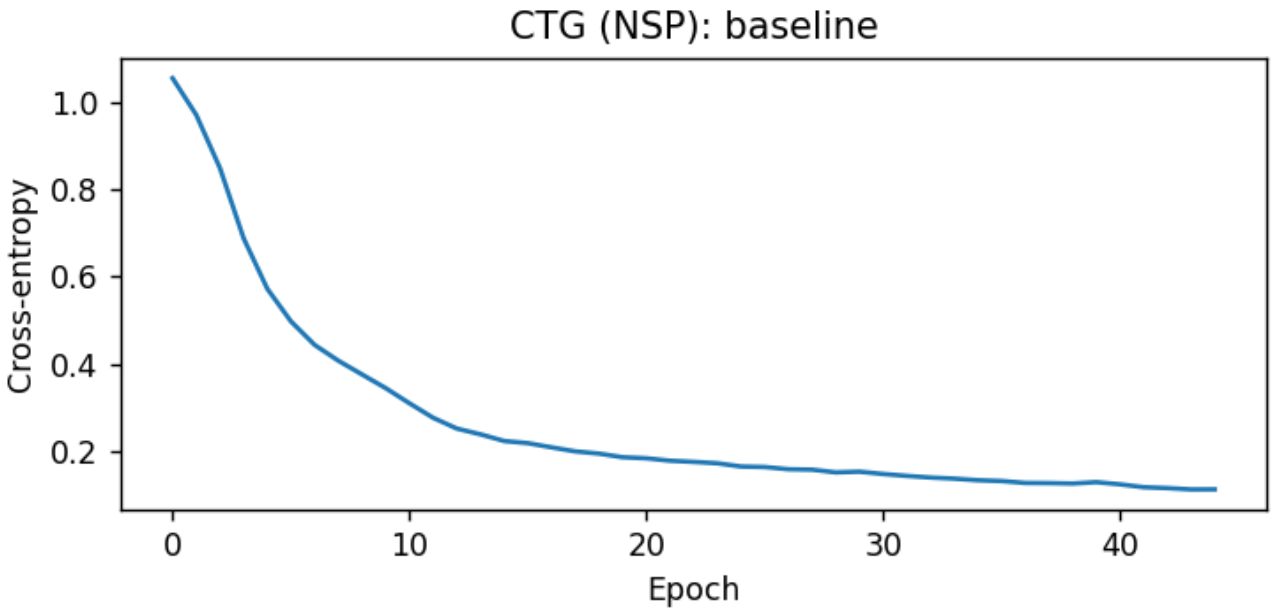


Рисунок. ctg baseline loss

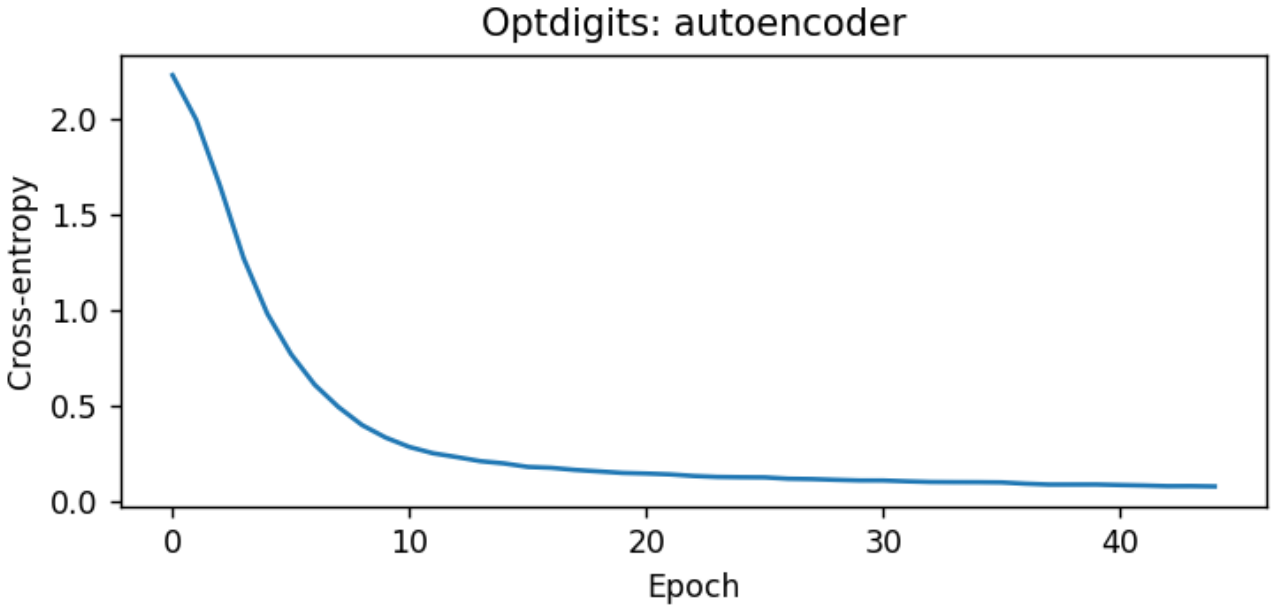


Рисунок. optdigits autoencoder loss

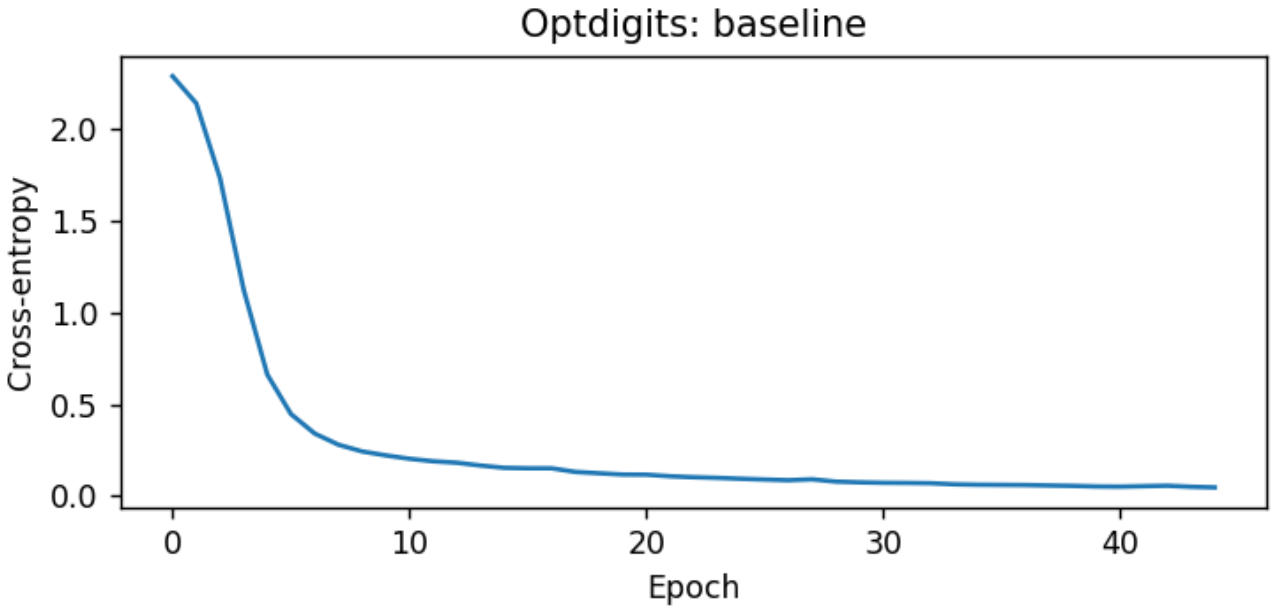


Рисунок. optdigits baseline loss

Выводы о проделанной работе:

1. CTG (NSP): baseline: accuracy=0.8929, macro-F1=0.7943, autoencoder: accuracy=0.9098, macro-F1=0.8225.

2. Optdigits: baseline: accuracy=0.9527, macro-F1=0.9528, autoencoder: accuracy=0.9444, macro-F1=0.9444.

3. Сравнение проведено без изменения архитектуры, тестовых индексов и целевой переменной. В Cardiotocography используется NSP (3 класса), CLASS исключен из признаков.

4. Для Optdigits обучение проводилось на тренировочной части .tra, оценка — на отложенной .tes.

Код программы: ИАД_3.py

```
"""ИАД ЛР №3, вариант 5: обучение DNN с AE-предобучением и без него, NSP и optdigits."""
from pathlib import Path
from NN_common import datasets,train,report
ROOT=Path(__file__).resolve().parent
ans={}
for name,(X,y,tr,te) in datasets(ROOT).items():
 ans[name]=[train(X,y,tr,te,method,ROOT,name) for method in ['baseline','autoencoder']]
report(ans,ROOT)
```

Код программы: NN_common.py

```
"""Общий воспроизводимый эксперимент для ЛР 3-4, вариант 5."""
from pathlib import Path
import json,random,time
import numpy as np,pandas as pd,torch
from torch import nn
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,f1_score,confusion_matrix,classification_report
from sklearn.neural_network import BernoulliRBM
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
SEED=42


def seeds():
 random.seed(SEED);np.random.seed(SEED);torch.manual_seed(SEED);torch.set_num_threads(2)


def datasets(root):
 ctg=pd.read_csv(root/'data/CTG.csv')
 X=ctg.drop(columns=['NSP','CLASS']).values.astype('float32');y=ctg.NSP.to_numpy().astype(int)-1
 train,test=train_test_split(np.arange(len(y)),test_size=.25,stratify=y,random_state=SEED)
 scale=StandardScaler().fit(X[train]);X=scale.transform(X).astype('float32')

a=pd.read_csv(root/'data/optdigits.tra',header=None).to_numpy();b=pd.read_csv(root/'data/optdigits.tes',header=None).to_numpy(
)
 digits=np.vstack([a,b]);XX=digits[:,:-1].astype('float32')/16;yy=digits[:,-1].astype(int)
 tr=np.arange(len(a));te=np.arange(len(a),len(digits))
 return {'CTG (NSP)':(X,y,train,test),'Optdigits':(XX,yy,tr,te)}


class DNN(nn.Module):
 def __init__(self,features,n_classes):

super().__init__();self.layers=nn.Sequential(nn.Linear(features,64),nn.ReLU(),nn.Linear(64,32),nn.ReLU(),nn.Linear(32,16),nn.R
eLU(),nn.Linear(16,n_classes))
 def forward(self,z):return self.layers(z)


def ae_pretrain(model,X,train_idx,epochs=13):
 # Отдельные автоэнкодеры для трёх последовательных весовых слоёв.
 z=torch.tensor(X[train_idx]);weights=[(0,64),(2,32),(4,16)]
 for position,out_dim in weights:
  encoder=nn.Linear(z.shape[1],out_dim);decoder=nn.Linear(out_dim,z.shape[1])
  auto=nn.Sequential(encoder,nn.ReLU(),decoder)
  optimizer=torch.optim.Adam(auto.parameters(),lr=.003)
  for _ in range(epochs):
   for mini in z.split(256):
    optimizer.zero_grad();pred=auto(mini);loss=nn.functional.mse_loss(pred,mini);loss.backward();optimizer.step()
  with torch.no_grad():
   layer=model.layers[position];layer.weight.copy_(encoder.weight);layer.bias.copy_(encoder.bias)
   z=torch.relu(encoder(z)).detach()


def rbm_pretrain(model,X,train_idx,epochs=12):
 # BernoulliRBM: вход масштабируется в [0,1]. После слоя используем средние активации.
 z=X[train_idx].astype('float64')
 mins=z.min(axis=0);ranges=z.max(axis=0)-mins;ranges[ranges==0]=1;z=np.clip((z-mins)/ranges,0,1)
 for position,n_hidden in [(0,64),(2,32),(4,16)]:
  rbm=BernoulliRBM(n_components=n_hidden,learning_rate=.02,batch_size=64,n_iter=epochs,random_state=SEED,verbose=0)
  rbm.fit(z)
  layer=model.layers[position]
  # RBM имеет сигмоидную активацию, DNN использует ReLU; переносим веса как инициализацию.
  with torch.no_grad():
   layer.weight.copy_(torch.tensor(rbm.components_,dtype=torch.float32))
   layer.bias.copy_(torch.tensor(rbm.intercept_hidden_,dtype=torch.float32))
  z=rbm.transform(z)


def train(X,y,tr,te,method,root,label,epochs=45):
 seeds();n_classes=len(np.unique(y));model=DNN(X.shape[1],n_classes)
 if method=='autoencoder':ae_pretrain(model,X,tr)
 elif method=='RBM':rbm_pretrain(model,X,tr)
 optimizer=torch.optim.Adam(model.parameters(),lr=.002)
 trainx=torch.tensor(X[tr]);trainy=torch.tensor(y[tr],dtype=torch.long)
 losses=[];start=time.perf_counter()
 for ep in range(epochs):
```

```
  perm=torch.randperm(len(tr));epoch_loss=0.0
  for ix in perm.split(256):

optimizer.zero_grad();output=model(trainx[ix]);loss=nn.functional.cross_entropy(output,trainy[ix]);loss.backward();optimizer.s
tep();epoch_loss+=float(loss.detach())*len(ix)
  losses.append(epoch_loss/len(tr))
 with torch.no_grad():p=model(torch.tensor(X[te])).argmax(dim=1).numpy()
 outcome={'method':method,'dataset':label,'samples_train':len(tr),'samples_test':len(te),'epochs':epochs,
          'accuracy':float(accuracy_score(y[te],p)),'f1_macro':float(f1_score(y[te],p,average='macro')),
          'f1_weighted':float(f1_score(y[te],p,average='weighted')),
          'confusion_matrix':confusion_matrix(y[te],p).tolist(),
          'classification_report':classification_report(y[te],p,zero_division=0),
          'training_seconds':round(time.perf_counter()-start,2),'training_loss':losses}
 folder=root/'results';folder.mkdir(exist_ok=True)
 key='ctg' if label.startswith('CTG') else 'optdigits'
 torch.save(model.state_dict(),folder/f'{key}_{method}.pt')
 plt.figure(figsize=(6,3));plt.plot(losses);plt.title(f'{label}: {method}');plt.xlabel('Epoch');plt.ylabel('Cross-
entropy');plt.tight_layout();plt.savefig(folder/f'{key}_{method}_loss.png',dpi=125);plt.close()
 return outcome


def report(results,root):
 (root/'results'/'metrics.json').write_text(json.dumps(results,indent=2,ensure_ascii=False),encoding='utf8')
 for dataset,rows in results.items():
  print(dataset)
  for m in
rows:print(m['method'],'accuracy=',round(m['accuracy'],4),'f1_macro=',round(m['f1_macro'],4),'time_s=',m['training_seconds'])
```

Исходный код, данные и графики также приложены к лабораторной работе в директории src/.